GPU, CUDA, PyTorch 환경 설정 가이드

> **대상**: GPU 환경에서의 개발이 처음인 학생
> **목표**: 내 PC에서 GPU를 활용한 AI 모델 실행 환경을 올바르게 구성할 수 있다.
> **환경**: Windows OS + NVIDIA GPU 기준으로 작성되었습니다.

이런 상황이라면?

상황	바로가기
처음 설치할 때	5. PyTorch 설치 및 재설치 방법
내 PC에 GPU가 있는지 모를 때	2. 내 PC에서 GPU를 사용할 수 있는지 확인하기
GPU가 있는데 인식이 안 될 때	6. 자주 발생하는 문제와 해결
nvidia-smi 출력을 읽고 싶을 때	3. nvidia-smi 사용법
CUDA/PyTorch 버전을 어떻게 맞추는지	4. CUDA와 PyTorch 버전 호환성

---

1. 기본 개념 이해

GPU란?

GPU(Graphics Processing Unit) 는 원래 게임이나 영상의 그래픽 처리를 위해 만들어진 장치입니다. CPU가 복잡한 작업을 **순차적으로** 처리하는 데 특화되어 있다면, GPU는 단순한 연산 수천 개를 **동시에(병렬로)** 처리하는 데 특화되어 있습니다.

AI/딥러닝에서는 행렬 곱셈 같은 단순 반복 연산이 대량으로 필요하기 때문에, GPU가 CPU보다 **수십~수백 배** 빠릅니다.

```
[CPU]                          [GPU]
강력한 코어 4~16개              약한 코어 수천 개
복잡한 작업 1개씩 빠르게 처리    단순한 작업 수천 개를 동시에 처리
일반 프로그래밍에 적합          AI/딥러닝 연산에 적합
```

신경망으로 이해하는 GPU의 병렬 처리

아래와 같은 신경망에서 Input Layer(5개 노드) → Hidden Layer 1(6개 노드) 사이에는 **5 x 6 = 30개의 연결(가중치 곱셈)**이 필요합니다. 이 선 하나하나가 곱셈 연산입니다.

```
[Input]     [Hidden 1]     [Hidden 2]     [Output]
  ●──────────●
  ●──────────●              ●              ●
  ●──────────●              ●              ●
  ●──────────●              ●              ●
```

```
    ●──────────●
               ●
   (5개 노드)  (6개 노드)    (6개 노드)     (4개 노드)
```

CPU vs GPU 처리 방식 차이:

```
Input → Hidden Layer 1 사이의 연산 (30개 곱셈)

CPU:  연결1 → 연결2 → 연결3 → ... → 연결30   (하나씩 순차 처리)
      ──────────────────────────────────────→ 시간

GPU:  연결1  —→ ┐
      연결2  —→ │
      연결3  —→ ├→ 완료!   (수천 개 코어가 동시에 처리)
      ...    —→ │
      연결30 —→ ┘
      ──────────→ 시간 (훨씬 짧음)
```

구분	CPU	GPU
Input → Hidden 1 (30개 곱셈)	하나씩 순서대로	**30개를 동시에**
Hidden 1 → Hidden 2 (36개 곱셈)	하나씩 순서대로	**36개를 동시에**
Hidden 2 → Output (24개 곱셈)	하나씩 순서대로	**24개를 동시에**

> **핵심 포인트:**
>
> - **레이어 내 연산은 병렬** — 같은 레이어 사이의 연결(곱셈)들은 서로 독립적이므로 GPU가 동시에 처리 가능
> - **레이어 간은 순차** — Hidden Layer 1의 결과가 나와야 Hidden Layer 2를 계산할 수 있음 (의존 관계)
> - 이것이 바로 **행렬 곱셈** — `[5x1 입력 벡터] × [5x6 가중치 행렬]`이며, 행렬 곱셈 안의 개별 곱셈·덧셈들이 독립적이라 GPU가 빠른 것

VRAM이란?

VRAM(Video RAM) 은 GPU 전용 메모리입니다. 일반 RAM(시스템 메모리)과는 별개로, GPU가 데이터를 처리할 때 사용하는 **GPU만의 메모리 공간**입니다.

구분	RAM (시스템 메모리)	VRAM (GPU 메모리)
사용 주체	CPU	GPU
일반적인 용량	8~32 GB	4~24 GB
용도	프로그램 실행, OS 운영	AI 모델 로드, 그래픽 처리
확인 방법	작업 관리자 → 성능 → 메모리	`nvidia-smi` 또는 작업 관리자 → 성능 → GPU

> AI 모델은 VRAM에 올라가야 GPU로 연산할 수 있습니다. VRAM이 부족하면 `CUDA out of memory` 에러가 발생합니다.

전용 GPU vs 공유 GPU(내장 GPU)

작업 관리자에서 GPU 메모리를 보면 **전용 GPU 메모리**와 **공유 GPU 메모리** 두 가지가 표시됩니다.

```
전용 GPU 메모리    0.5 / 12.0 GB   ← GPU 카드에 달린 물리 메모리 (VRAM)
공유 GPU 메모리    0.1 /  8.0 GB   ← 시스템 RAM에서 빌려 쓰는 것 (느림)
```

이 둘은 물리적으로 다른 장치입니다.

구분	전용 GPU (외장)	공유 GPU (내장)
장치	NVIDIA/AMD 별도 그래픽 카드	CPU 안에 포함된 그래픽 칩
메모리	**자체 VRAM** (4~24GB)	시스템 **RAM을 빌려서** 사용
성능	높음 (AI/게임에 적합)	낮음 (웹, 문서 작업 수준)
대표 예시	NVIDIA RTX 3060, RTX 4070	Intel UHD Graphics, Intel Iris Xe
AI 실습	CUDA로 GPU 가속 가능	CUDA 사용 불가 (CPU로 실행)

- **전용 GPU 메모리**: GPU 칩 바로 옆에 붙어 있어서 **매우 빠름** → AI 모델이 여기에 올라감
- **공유 GPU 메모리**: 시스템 RAM의 일부를 GPU가 빌려 쓰는 것 → VRAM이 부족할 때 넘치는 데이터가 여기로 가지만 **훨씬 느림**

> 내장 GPU만 있는 PC에서도 일부 실습은 가능합니다. 다만 CUDA를 사용할 수 없으므로 `device = -1`(CPU 모드)로 동작하고, 속도가 느려집니다.

Windows PC vs Apple Silicon Mac — GPU 구조의 차이

Mac(Apple Silicon)은 CUDA 없이도 GPU 가속이 편합니다. 이는 **하드웨어 메모리 설계 자체가 다르기 때문**입니다.

> **NPU(Neural Processing Unit)란?** AI 연산에 특화된 전용 프로세서입니다. GPU가 범용 병렬 연산을 처리한다면, NPU는 **AI 모델 추론(예: 이미지 인식, 음성 인식)만을 위해 설계**된 칩입니다. Apple Silicon의 Neural Engine, Intel/Qualcomm의 NPU 등이 대표적입니다. GPU보다 적은 전력으로 AI 연산을 처리할 수 있어, 스마트폰이나 노트북 같은 배터리 기기에서 주로 활용됩니다. 아래 다이어그램에서 Apple Silicon에 CPU, GPU와 함께 NPU가 포함되어 있는 것을 확인할 수 있습니다.
>
> | 구분 | CPU | GPU | NPU |
> |---|---|---|---|
> | 설계 목적 | 범용 연산 | 병렬 연산 (그래픽/AI) | AI 추론 전용 |
> | 코어 수 | 4~16개 (강력) | 수천 개 (약한) | AI 전용 코어 |
> | 전력 소비 | 보통 | 높음 | 낮음 |
> | 대표 용도 | 프로그래밍, OS | AI 학습, 게임 | 얼굴 인식, 음성 인식 |

```
[일반 PC (Windows)]                    [Apple Silicon Mac (M1/M2/M3/M4)]

┌──────────┐      ┌──────────┐          ┌──────────────────────┐
│   CPU    │      │   GPU    │          │    CPU + GPU + NPU   │
│          │      │          │          │                      │
│  RAM     │ ↔    │  VRAM    │          │   통합 메모리 (RAM)   │
│ 16~32GB  │전송  │ 4~24GB   │          │      16~128GB        │
└──────────┘      └──────────┘          └──────────────────────┘

  별도 메모리      별도 메모리              하나의 메모리를 공유
  데이터를 GPU로 복사해야 함              복사 없이 바로 접근
```

구분	Windows + NVIDIA	Apple Silicon Mac
GPU 가속 기술	CUDA (NVIDIA 전용)	MPS (Metal Performance Shaders)
메모리 구조	CPU RAM과 VRAM이 **분리**	CPU와 GPU가 메모리를 **공유 (통합 메모리)**
VRAM 제한	GPU 카드에 달린 만큼 (8GB, 12GB 등)	시스템 RAM 전체를 GPU가 사용 가능
데이터 전송	RAM → VRAM 복사 필요 (병목)	복사 없이 바로 접근 (빠름)
설치 편의성	CUDA 버전 호환성을 맞춰야 함	`pip install torch`만 하면 끝
드라이버 관리	nvidia-smi, 버전 확인 필요	별도 관리 불필요 (OS에 내장)

Mac이 편한 3가지 이유:

1. **버전 호환성 문제가 없음** — CUDA 버전, 드라이버 버전을 맞출 필요 없이 PyTorch만 설치하면 MPS가 자동으로 동작 (단, 일부 연산이나 라이브러리는 MPS를 아직 지원하지 않을 수 있음)
2. **VRAM 부족 문제가 적음** — 통합 메모리라서 RAM 32GB Mac이면 이론적으로 GPU도 해당 메모리를 활용 가능. 다만 OS와 다른 앱이 사용하는 영역이 있고, macOS는 시스템 안정성을 위해 하나의 프로세스가 사용할 수 있는 메모리 할당량을 전체 RAM의 약 75~80%로 제한하므로 RAM 전체를 다 쓰지는 못함. 그래도 Windows에서 `CUDA out of memory`가 나는 모델이 Mac에서는 돌아가는 경우가 있음
3. **설정이 단순함** — `nvidia-smi` 확인, `--index-url` 지정 등의 과정이 전혀 필요 없음

그렇다면 Mac이 항상 좋은가?

아닙니다. 동일 세대·동급 가격대 기준에서 **순수 연산 속도**는 NVIDIA GPU가 훨씬 빠릅니다.

항목	NVIDIA RTX 4070 (12GB VRAM)	M3 Pro (18GB 통합 메모리)
AI 추론 속도	매우 빠름	보통
학습(Training) 속도	매우 빠름	느림
설정 편의성	복잡함	간편함
큰 모델 로드	VRAM 12GB 한계	RAM까지 활용 가능
업계 표준	CUDA가 사실상 표준	지원 범위가 제한적

> **정리**: Mac은 "설정이 쉽고 메모리 제약이 적어서 편하다", NVIDIA는 "설정은 복잡하지만 속도가 빠르고 업계 표준이다". 이번 수업에서는 교실 PC(NVIDIA GPU)를 기준으로 실습합니다.

CUDA란?

CUDA(Compute Unified Device Architecture) 는 NVIDIA가 만든 **GPU 프로그래밍 플랫폼**입니다. 쉽게 말해, Python 코드에서 GPU를 사용할 수 있게 해주는 **다리(인터페이스)** 역할을 합니다.

```
Python 코드 (PyTorch)
        │
        ▼
    CUDA (GPU 프로그래밍 인터페이스)
        │
        ▼
    NVIDIA GPU (실제 연산 수행)
```

> **중요**: CUDA는 **NVIDIA GPU 전용**입니다. AMD GPU나 Intel 내장 그래픽에서는 사용할 수 없습니다.

PyTorch(torch)란?

PyTorch는 Facebook(Meta)이 만든 **딥러닝 프레임워크**입니다. AI 모델을 만들고, 학습시키고, 실행하는 데 사용하는 Python 라이브러리입니다.

```python
import torch

# CPU에서 연산
x = torch.tensor([1.0, 2.0, 3.0])

# GPU에서 연산 (CUDA 필요)
x = torch.tensor([1.0, 2.0, 3.0]).cuda()
```

PyTorch는 설치할 때 **CUDA 런타임을 자체 포함(bundled)** 합니다. 대부분의 실습에서는 PyTorch wheel에 포함된 CUDA 런타임만으로 충분하며, 별도로 CUDA Toolkit을 설치할 필요가 없습니다. (CUDA Toolkit 별도 설치가 필요한 경우는 커스텀 CUDA 커널 개발이나 특정 컴파일 확장 라이브러리를 사용할 때뿐입니다.)

> **참고 — PyTorch에서 GPU를 사용하는 두 가지 방식:**
>
> - **HuggingFace Pipeline**: `device=0`(GPU) 또는 `device=-1`(CPU) 파라미터로 지정 — 이번 수업에서 사용하는 방식
> - **PyTorch 직접 사용**: `.cuda()` 또는 `.to('cuda')` 메서드로 텐서/모델을 GPU로 이동
>
> ```python
> # HuggingFace Pipeline 방식
> classifier = pipeline("sentiment-analysis", device=0)
>
> # PyTorch 직접 사용 방식
> x = torch.tensor([1.0, 2.0]).to('cuda')
> ```

전체 구조 정리

```
┌──────────────────────────────────────────────────┐
│  Python 코드                                      │
│  from transformers import pipeline                │
│  classifier = pipeline("sentiment-analysis",      │
│                        device=0)  ← GPU 사용!     │
├──────────────────────────────────────────────────┤
│  PyTorch (torch)                                  │
│  - AI 모델 연산을 담당하는 딥러닝 프레임워크        │
│  - CUDA 런타임을 자체 포함하고 있음                 │
├──────────────────────────────────────────────────┤
│  NVIDIA 드라이버                                  │
│  - GPU와 소프트웨어를 연결하는 드라이버             │
│  - nvidia-smi로 상태 확인 가능                    │
├──────────────────────────────────────────────────┤
│  NVIDIA GPU (하드웨어)                            │
│  - 실제 연산을 수행하는 물리 장치                   │
│  - VRAM에 모델을 올려서 빠르게 처리                 │
└──────────────────────────────────────────────────┘
```

---

2. 내 PC에서 GPU를 사용할 수 있는지 확인하기

Step 1: NVIDIA GPU가 있는지 확인

방법 A: 작업 관리자

1. `Ctrl + Shift + Esc`로 작업 관리자 열기
2. **성능** 탭 클릭
3. 왼쪽 목록에서 **GPU** 항목 확인
4. GPU 이름에 **NVIDIA**가 포함되어 있으면 사용 가능

```
☑ 사용 가능: NVIDIA GeForce RTX 3060, NVIDIA RTX 4070, NVIDIA GTX 1660 등
❌ 사용 불가: Intel UHD Graphics, AMD Radeon RX 580 등
```

방법 B: 장치 관리자

1. `Windows + X` → 장치 관리자
2. **디스플레이 어댑터** 펼치기
3. NVIDIA GPU가 목록에 있는지 확인

방법 C: 터미널

```
# NVIDIA 드라이버가 설치되어 있으면 GPU 정보가 출력됨
nvidia-smi
```

만약 `'nvidia-smi' is not recognized` 에러가 나오면:

- NVIDIA GPU가 없거나
- NVIDIA 드라이버가 설치되지 않은 상태
- 또는 nvidia-smi가 시스템 PATH에 등록되지 않은 경우 → 아래 경로에서 직접 실행해보세요:

```
# Windows 기본 경로
"C:\Windows\System32\nvidia-smi.exe"
```

Step 2: VRAM 용량 확인

작업 관리자 → 성능 → GPU에서 **전용 GPU 메모리** 항목을 확인합니다.

```
전용 GPU 메모리    0.5 / 12.0 GB   ← 12GB VRAM
공유 GPU 메모리    0.1 /  8.0 GB   ← 시스템 RAM에서 빌려 쓰는 것 (느림)
```

VRAM 용량	AI 실습 가능 여부
4 GB	소형 모델 가능 (DistilBERT 등)
6~8 GB	대부분의 실습 가능
12 GB 이상	대형 모델도 여유 있음

> 이번 5주차 실습에서 3개 모델을 모두 로드하면 약 3GB VRAM을 사용합니다.

Step 3: GPU가 없는 경우

GPU가 없어도 **CPU로 실행 가능**합니다. 다만 속도가 느립니다.

```
# 이 코드가 자동으로 CPU/GPU를 판단합니다
device = 0 if torch.cuda.is_available() else -1
```

환경	device 값	속도 (감성 분석 기준)
GPU (CUDA)	`0`	~0.1초
CPU	`-1`	~1~3초

> **주의**: `device = 0` / `device = -1` 숫자 표기는 **HuggingFace Pipeline 라이브러리의 전용 방식**입니다. 순수 PyTorch에서는 `"cuda"` / `"cpu"` 문자열을 사용합니다. `tensor.to(-1)`처럼 작성하면 에러가 발생하니 혼동하지 마세요.
>
> ```
> # HuggingFace Pipeline (이번 수업 방식)
> classifier = pipeline("sentiment-analysis", device=0)   # GPU
> classifier = pipeline("sentiment-analysis", device=-1)  # CPU
> ```

```
# 순수 PyTorch (참고)
x = torch.tensor([1.0]).to("cuda")  # GPU
x = torch.tensor([1.0]).to("cpu")   # CPU
```

---

3. nvidia-smi 사용법

`nvidia-smi`는 NVIDIA GPU의 상태를 확인하는 **터미널 명령어**입니다.

기본 실행

```
nvidia-smi
```

출력 읽는 법

```
+-----------------------------------------------------------------------------
--------+
| NVIDIA-SMI 535.183.01   Driver Version: 535.183.01   CUDA Version: 12.1
|
|                                                      ~~~~~~~~~~~~~~
|
|                                                      ↑ 드라이버가 지원하는 최대
CUDA 버전  |
|-----------------------------------------+----------------------+-------------
--------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile
Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util
Compute M. |
|=========================================+======================+=============
========|
|   0  NVIDIA GeForce RTX 3060 ...   Off | 00000000:01:00.0  On  |
N/A |
|  0%   42C    P8              15W / 170W |    512MiB / 12288MiB   |      0%
Default |
|                                         ~~~~~~~~~~  ~~~~~~~~~~
|
|                                         사용 중       전체 VRAM
|
+-----------------------------------------+----------------------+-------------
--------+
```

확인해야 할 3가지:

항목	위치	예시	의미

항목	위치	예시	의미
CUDA Version	오른쪽 상단	`12.1`	드라이버가 지원하는 최대 CUDA Toolkit 버전. PyTorch 설치 시 이 버전 이하를 선택해야 함
Memory-Usage	중간	`512MiB / 12288MiB`	현재 VRAM 사용량 / 전체 VRAM
GPU-Util	오른쪽	`0%`	GPU 사용률 (AI 추론 중에는 높아짐)

> **주의**: CUDA Version에 `12.1`이 표시되어 있다고 해서 **CUDA Toolkit 12.1이 PC에 설치되어 있다는 뜻이 아닙니다.** 이 숫자는 드라이버가 지원하는 "한계치"일 뿐입니다. 하지만 PyTorch는 CUDA 런타임을 자체 포함하고 있으므로, 이 숫자 이하의 cu 빌드를 설치하면 별도의 CUDA Toolkit 설치 없이 정상 작동합니다.

실시간 모니터링

```
# 10초마다 자동 갱신 (가장 간단)
nvidia-smi -l 10

# 종료: Ctrl + C
```

> AI 모델을 로드할 때 Memory-Usage가 증가하는 것을 직접 관찰해보세요.

---

4. CUDA와 PyTorch 버전 호환성

핵심 규칙

> **PyTorch에 포함된 CUDA 버전 ≤ nvidia-smi에 표시된 CUDA Version**

PyTorch는 설치할 때 특정 CUDA 버전용으로 빌드된 패키지를 받습니다. 이 버전이 드라이버가 지원하는 CUDA 버전보다 **높으면 GPU를 인식하지 못합니다.**

> 정확히는 NVIDIA 드라이버와 CUDA 런타임 사이의 호환성 문제이지만, **"nvidia-smi에 표시된 CUDA Version보다 높은 cu 빌드는 피한다"** 정도로 기억하면 충분합니다.

호환성 표

nvidia-smi CUDA Version	사용 가능한 PyTorch CUDA 빌드
12.8	cu128, cu126, cu124, cu121, cu118
12.6	cu126, cu124, cu121, cu118
12.4	cu124, cu121, cu118
12.1	cu121, cu118
11.8	cu118

> **가장 가까운 버전**을 선택하는 것이 성능상 좋습니다. 예를 들어 CUDA 12.1이면 `cu121`을 선택합니다.
>
> **주의**: PyTorch 버전마다 제공되는 CUDA 빌드가 다릅니다. 예를 들어 `cu126`은 특정 PyTorch 버전부터 제공됩니다. 설치 전에 PyTorch 공식 설치 페이지에서 원하는 PyTorch 버전에서 사용 가능한 CUDA 빌드를 확인하세요. 원하는 cu 빌드가 없다면 그보다 낮은 버전(예: cu121 → cu118)을 선택하면 됩니다.

호환되지 않는 버전을 설치하면?

```
import torch
print(torch.cuda.is_available())  # False ← GPU가 있는데도 False!
```

GPU가 분명히 있는데 `False`가 나온다면, **PyTorch CUDA 버전이 드라이버보다 높은 것**이 가장 흔한 원인입니다.

---

5. PyTorch 설치 및 재설치 방법

사전 준비: 가상환경 생성

PyTorch를 설치하기 전에 **가상환경(venv)** 을 먼저 만드는 것을 권장합니다. 시스템 Python에 직접 설치하면 다른 프로젝트와 패키지 버전이 충돌할 수 있습니다.

```
# 1. 프로젝트 폴더로 이동
cd 5_week

# 2. 가상환경 생성
python -m venv venv

# 3. 가상환경 활성화 (Windows)
venv\Scripts\activate

# 활성화되면 프롬프트 앞에 (venv)가 표시됩니다:
# (venv) C:\Users\...\5_week>

# 4. 이후 pip install 명령을 실행하면 가상환경 안에만 설치됩니다
```

항목	시스템 Python에 직접 설치	가상환경(venv) 사용
다른 프로젝트에 영향	있음 (버전 충돌 위험)	없음 (격리됨)
패키지 초기화	어려움	`venv` 폴더 삭제하면 끝
권장 여부	비추천	**추천**

> **팁**: 가상환경을 비활성화하려면 `deactivate` 명령어를 입력하세요. 터미널을 닫아도 자동으로 비활성화됩니다.

신규 설치

```
# 1. nvidia-smi로 CUDA 버전 확인
nvidia-smi
# → CUDA Version: 12.1 확인

# 2. 호환되는 버전으로 설치
pip install torch==2.6.0 torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu121
```

기존 패키지 제거 후 재설치

잘못된 버전을 설치했거나 GPU 인식이 안 될 때:

```
# 1. 기존 패키지 완전 제거
pip uninstall torch torchvision torchaudio -y

# 2. 올바른 버전으로 재설치
pip install torch==2.6.0 torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu121
```

GPU가 없는 경우 (CPU 전용)

```
pip install torch==2.6.0 torchvision torchaudio --index-url
https://download.pytorch.org/whl/cpu
```

> CPU 전용 버전을 설치하려면 `--index-url`에 `/whl/cpu`를 명시해야 합니다. 옵션 없이 설치하면 플랫폼에 따라 CUDA 포함 버전이 설치될 수 있습니다.

참고: Apple Silicon Mac (M1/M2/M3/M4)

```
pip install torch torchvision torchaudio
```

> Apple Silicon Mac은 NVIDIA GPU가 아니므로 CUDA를 사용할 수 없지만, **MPS(Metal Performance Shaders)** 를 통해 GPU 가속을 사용할 수 있습니다. 별도의 `--index-url` 옵션 없이 설치하면 MPS가 자동으로 지원됩니다.
>
> ```
> import torch
> print(torch.backends.mps.is_available())  # True면 MPS 사용 가능
> ```
>
> **주의**: 일부 연산이나 라이브러리는 MPS를 아직 지원하지 않을 수 있습니다. MPS에서 에러가 발생하면 `device=-1`(CPU)로 전환하여 실행하세요.

설치 확인

```
import torch

print(torch.__version__)            # 2.6.0+cu121
print(torch.cuda.is_available())    # True (GPU 사용 가능)
print(torch.cuda.get_device_name(0))  # NVIDIA GeForce RTX 3060 (GPU 이름)
```

출력	의미
`2.6.0+cu121`	PyTorch 2.6.0, CUDA 12.1 빌드
`2.6.0+cpu`	PyTorch 2.6.0, CPU 전용 (GPU 사용 불가)
`torch.cuda.is_available() → True`	GPU 정상 인식
`torch.cuda.is_available() → False`	GPU 미인식 (버전 불일치 또는 GPU 없음)

---

6. 자주 발생하는 문제와 해결

GPU가 있는데 `torch.cuda.is_available()`이 `False`

원인 1: PyTorch CUDA 버전이 드라이버보다 높음 (가장 흔함)

```
# 확인
nvidia-smi          # CUDA Version: 12.1
python -c "import torch; print(torch.__version__)"  # 2.6.0+cu124 ← 12.4 > 12.1 이므로 안 됨!

# 해결: 제거 후 올바른 버전 재설치
pip uninstall torch torchvision torchaudio -y
pip install torch==2.6.0 torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
```

원인 2: CPU 전용 버전이 설치됨

```
# 확인
python -c "import torch; print(torch.__version__)"  # 2.6.0+cpu ← GPU 빌드가 아님

# 해결: 제거 후 GPU 버전 재설치
pip uninstall torch torchvision torchaudio -y
pip install torch==2.6.0 torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
```

원인 3: NVIDIA 드라이버가 설치되지 않았거나, 오래된 버전임

> **중요: 현재 학교 PC 시스템에서는 드라이버 재설치 등의 작업을 진행하지 않습니다.** 아래 내용은 개인 PC 환경에서 참고하세요.

드라이버가 아예 없으면 `nvidia-smi`가 실행되지 않고, 드라이버가 오래되면 `nvidia-smi`의 CUDA Version이 낮아서 최신 PyTorch CUDA 빌드와 호환되지 않을 수 있습니다.

```
# 확인
nvidia-smi  # 'nvidia-smi' is not recognized → 드라이버 미설치
nvidia-smi  # CUDA Version: 11.4 → 드라이버가 오래됨 (cu121 사용 불가)
```

CUDA out of memory

GPU 메모리(VRAM)가 부족할 때 발생합니다.

```
# 1. 현재 VRAM 사용량 확인
nvidia-smi

# 2. Streamlit 서버 종료 (Ctrl+C) → VRAM 해제됨

# 3. VRAM이 해제되었는지 확인
nvidia-smi

# 4. 다시 실행
streamlit run app_multi.py
```

버전 확인 한눈에 보기

아래 코드를 Python에서 실행하면 현재 환경을 한 번에 확인할 수 있습니다:

```
import torch

print("=" * 50)
print(f"PyTorch 버전:  {torch.__version__}")
print(f"CUDA 사용 가능: {torch.cuda.is_available()}")

if torch.cuda.is_available():
    print(f"CUDA 버전:     {torch.version.cuda}")
    print(f"GPU 이름:      {torch.cuda.get_device_name(0)}")
    print(f"GPU 수:        {torch.cuda.device_count()}")
    props = torch.cuda.get_device_properties(0)
    vram = getattr(props, 'total_memory', getattr(props, 'total_mem', 0)) /
1024**3
    print(f"VRAM 용량:     {vram:.1f} GB")
else:
    print("⚠ GPU를 사용할 수 없습니다.")
    print("  - NVIDIA GPU가 없거나")
    print("  - PyTorch CUDA 버전과 드라이버가 호환되지 않습니다.")
    print(f"  - 현재 설치된 PyTorch: {torch.__version__}")
    if "+cpu" in torch.__version__:
        print("  - CPU 전용 버전이 설치되어 있습니다. GPU 버전으로 재설치하세요.")
print("=" * 50)
```

요약 — GPU 환경 설정 체크리스트

#	단계	명령어	확인 사항
1	NVIDIA GPU 확인	작업 관리자 → 성능 → GPU	GPU 이름에 "NVIDIA" 포함
2	드라이버 CUDA 버전 확인	`nvidia-smi`	오른쪽 상단 "CUDA Version" 숫자
3	호환되는 PyTorch 설치	`pip install torch==2.6.0 ... --index-url .../cu121`	CUDA Version 이하의 cu 빌드 선택
4	설치 확인	`python -c "import torch; print(torch.cuda.is_available())"`	`True` 출력
5	VRAM 모니터링	`nvidia-smi -l 10`	AI 모델 로드 시 Memory-Usage 증가 확인